



ENGINEERING DOCUMENTATION

Architecture

How Weave is put together, and — just as importantly — why it is put together this way.

Real

A commitment exists only where somebody accepted it, out loud.

Owned

Every item belongs to exactly one person, resolved from the platform.

Private

Nobody can see anybody else's work. Isolation is structural.

Explainable

Every claim can be traced to the sentence that produced it.

Four properties, and the structure that protects them

Every structural choice in this document exists to protect one of four properties: that a commitment is real, that it belongs to exactly one person, that nobody can see anybody else's work, and that the system can explain itself. Where a choice trades something away, it is named rather than hidden.

THE GOVERNING RULE

Product invariants live in Python, never in a prompt. A prompt is guidance that degrades under adversarial input; a validator holds for every input, including ones nobody imagined. Each invariant in this document is enforced in code and pinned by a named test — 265 of them, hermetic, with no network, no cloud credentials and no model calls.

Real

Only an explicit acceptance — or an explicit hand-off — becomes work. Silence stays silence, and an accepted item must carry the turn that proves it.

Owned

Owners resolve against the Meet attendee list with a confidence floor. Below it, Weave refuses rather than guesses.

Private

Enrichment runs once per owner, in a fresh session, as that owner, with access rights inside the query.

Explainable

Every commitment, edge and ranking cites the fact it came from.

WHAT THIS DOCUMENT COVERS

01	Design decisions and rationale	seven choices, each with its discarded alternative
02	The system at a glance	zones, identities and every hop between them
03	The end-to-end flow	one meeting, from transcript to card
04	The two-phase agent pipeline	the capability boundary at the core of the design
05	The context framework	owner-scoped retrieval, and the broker
06	The commitment graph	mentions folded into memory, edges only where stated
07	Prioritisation	arithmetic you can argue with
08	The Google Chat surface	one DM: onboarding, delivery, conversation
09	Interfaces, topology and packages	what runs where, and as whom

Design decisions

Seven decisions shape everything else. Each is stated with the alternative it was chosen over, because a decision without a discarded alternative is just a description.

1 Split the pipeline in two, and make the split a capability boundary

DECISION

Extraction reads the entire transcript and holds *no* context tools. Enrichment holds the context tool and runs once per owner, in a fresh session, as that owner.

INSTEAD OF

One agent that reads the transcript and looks things up as it goes.

WHY

A single agent would, at one moment, hold both everybody's words and the ability to search somebody's private files. That is precisely the combination that leaks. Splitting the phases means no single reasoning step ever holds both capabilities, so isolation stops depending on the model behaving and becomes a property of the wiring. It costs an extra model round trip per owner; that is a price worth paying for a guarantee that survives adversarial input.

2 Enforce product invariants in Python, never in a prompt

DECISION

“Only accepted work counts”, “an accepted item must cite its turn”, “results must belong to the owner” are pydantic validators and explicit filters. The prompt explains them; the code enforces them.

INSTEAD OF

Careful prompt instructions plus trust in the model.

WHY

A prompt is guidance that degrades under unusual or adversarial input; a validator holds for every input. It also makes the rules reviewable and testable — each invariant has a named test — rather than buried in prose. If a rule can only be expressed in a prompt string, it is not yet a rule.

3 Take identity from the Meet API, and fail closed

DECISION

Owners resolve against the real attendee list with a confidence floor. Below it — or for anyone who was not actually in the meeting — the owner gets an unenriched bundle and **no search at all**.

INSTEAD OF

Trusting the name the model returned, or falling back to a best guess.

WHY

A model asked “who is Sarah?” always produces a plausible answer. Acting on a wrong one routes a person's work *and* their private context to somebody else — the worst failure the system can have. Refusing to enrich is a visible, harmless degradation; guessing is an invisible, harmful one.

4 Put access rights inside the query, not around it

DECISION

Every context read carries the principal's ACL as a query predicate (`visible_to array_contains <email>`), so the database itself cannot return anything else.

INSTEAD OF

Fetching results and filtering them afterwards in application code.

WHY

Post-filtering is one forgotten branch away from a leak, and the unfiltered rows exist in memory in the meantime. A predicate cannot be forgotten: a query without it returns nothing rather than everything. The echo gate then re-checks the returned set as a second, independent layer.

5 Derive dependencies only from preconditions people actually spoke

DECISION

A `blocked_by` edge exists only where somebody stated the precondition out loud. Topical similarity never creates one.

INSTEAD OF

Inferring dependencies from semantic similarity between items.

WHY

The graph exists to answer “what should I do first”. A guessed edge corrupts exactly that answer, and does it invisibly — the ordering still looks confident. Fewer, true edges beat more, plausible ones, and every edge can be traced to a sentence.

6 Read-only by construction, with no write path at all

DECISION

There is no tool anywhere in the agent surface that writes to Drive, Tasks, Calendar or any other work system. Weave writes only its own derived records.

INSTEAD OF

Write access that is disabled by configuration or guarded by a permission check.

WHY

This is the entire prompt-injection story. A transcript is untrusted text written by whoever was in the room. With no write capability in the process, the best outcome for a malicious transcript is a bad suggestion on a card a human reads. Absent beats disabled: a capability that does not exist cannot be re-enabled by a mistake.

7 Deliver into the conversation people already have

DECISION

Weave has no application of its own. It lives in a Google Chat direct message: the same DM carries the install that onboards you, the card after each meeting, and the copilot you ask questions of.

INSTEAD OF

A Weave dashboard, inbox or task board.

WHY

The problem is that people have too many demands on their attention; a tool that adds another destination to check would work against its own purpose. Putting delivery and conversation in one DM also means the card and the answer are drawn from the identical judgement function, so they can never disagree about the facts — which would be worse than either being absent.

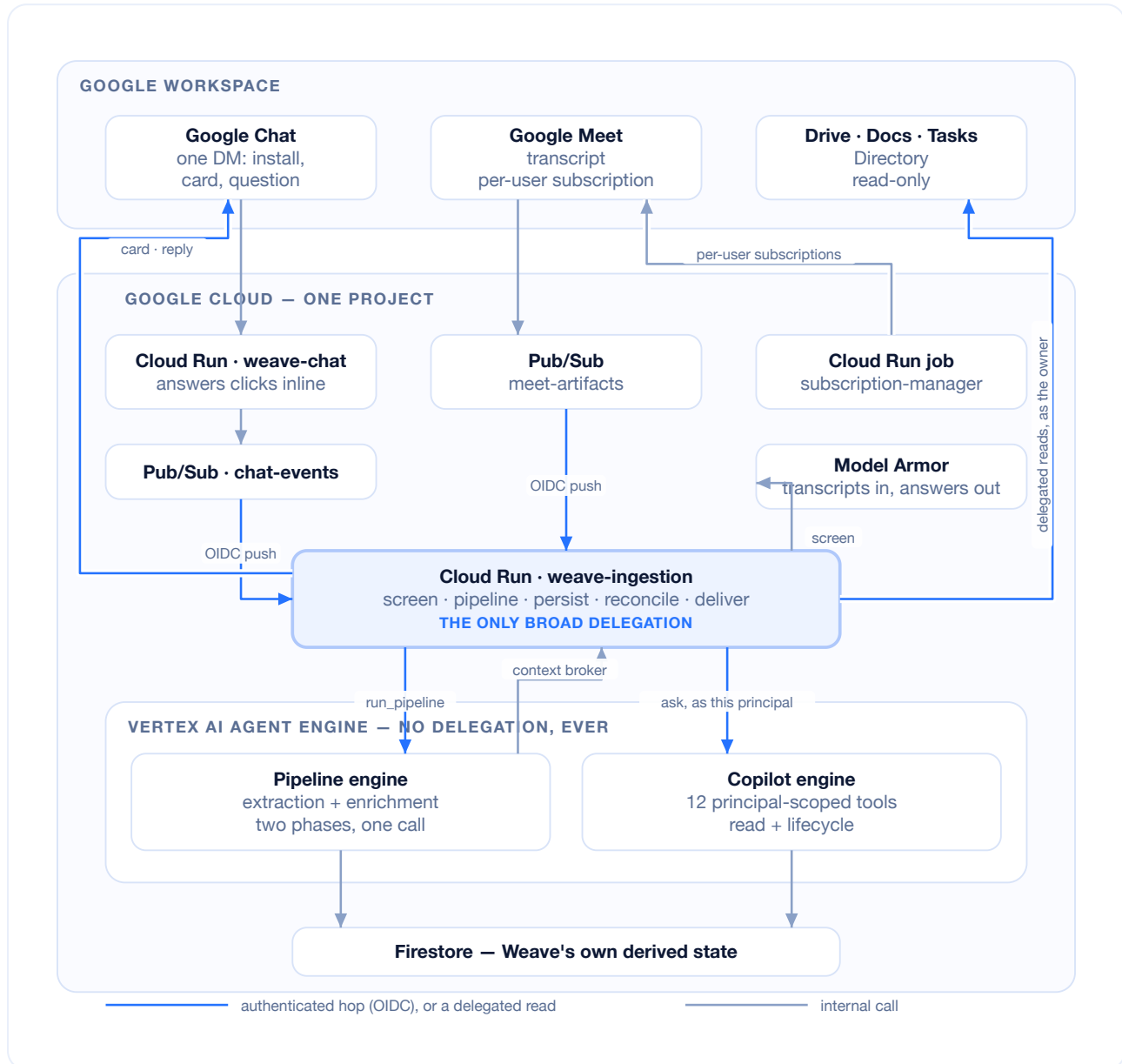
• Consequences worth naming

Good architecture is honest about what it trades away. Each of these is a deliberate floor, not an oversight.

CHOICE	WHAT IT COSTS
Two phases	An extra model call per owner, and a fan-out to orchestrate
Fail-closed identity	Some legitimate owners get an unenriched bundle rather than a guess
Spoken preconditions only	Real dependencies nobody articulated are not in the graph
No write path	Weave can tell you what you owe; closing the loop stays in your own tools
DM-only delivery	Nothing is posted to a shared space, so there is no team view
One project, no shared state	No cross-tenant view, and no organisation-wide reporting

The system at a glance

Two zones, one GCP project, and a single service that holds delegation. Everything else either has no Workspace reach at all, or reaches Workspace only through that service.



Zones and hops. Google Chat sits on the left because the same direct message carries all three interactions — the install that onboards, the card that delivers, and the question that gets answered. **weave-ingestion** is the hinge: it is the only identity in the system holding broad domain-wide delegation, and every delegated read is executed as the person whose context is being read. The Agent Engine deployments hold none, ever — when enrichment needs a Drive file, it asks ingestion.

Everything lives in **one GCP project**. There is no shared state outside it, and deleting the project removes the entire system.

The end-to-end flow

One meeting, from the moment a transcript exists to the moment a card lands — and the failure semantics that decide what a retry means. The full sequence is overleaf.

Failure semantics are part of the contract

A **200** acks the message — processed, duplicate, malformed or blocked are all terminal outcomes. A **500** leaves the message for Pub/Sub to redeliver. Anything unretryable is acked so it cannot poison the subscription, and anything a retry could fix marks the meeting **failed**, which is what makes it re-claimable.

Exactly once

A Firestore lease claims each conference id before work begins; a duplicate is acked without processing.

Atomic

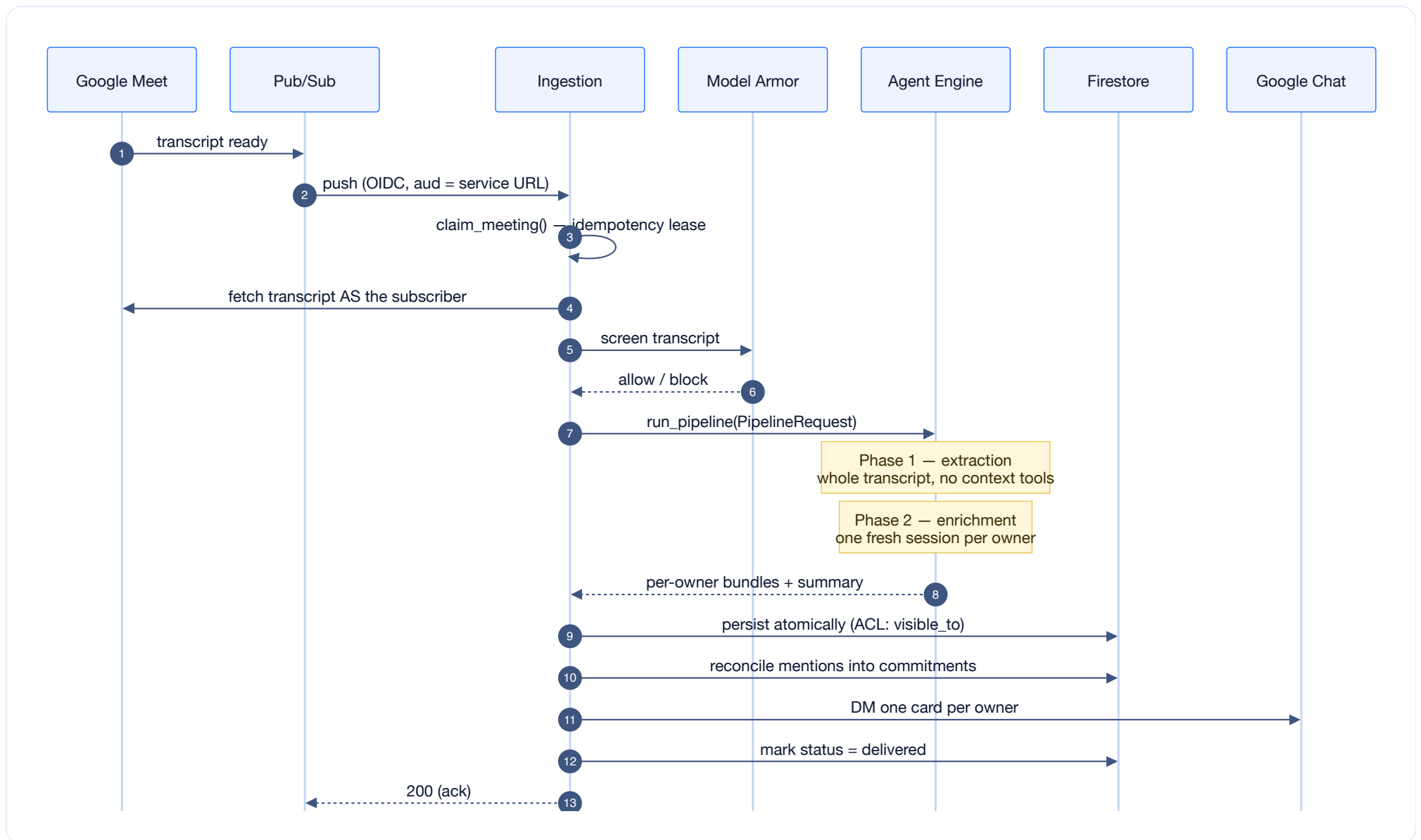
The summary and every action item are written in one batch, size checked in advance, so history can never disagree with its summary.

Contained

One owner's enrichment failure costs that owner their context, not the meeting. Reconciliation failure is recorded and delivery proceeds.

READ AS THE RIGHT PERSON

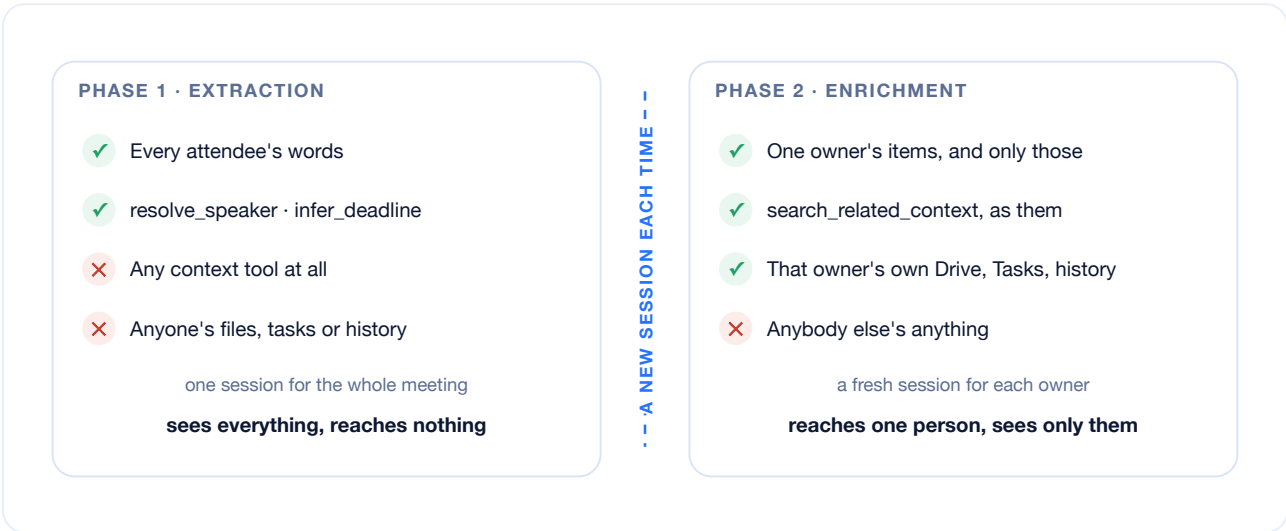
The Meet fetch impersonates the user whose subscription produced the event, read from the CloudEvent subject: a conference record is visible only to that conference's participants, so a single fixed subject would restrict the system to one person's meetings. When the subscriber cannot be determined the event fails with the full attribute set logged, rather than reading as the wrong account.



A meeting, from transcript to card. Steps 1–6 establish that this is a meeting Weave has not already processed, read as the person entitled to read it, and safe to hand to a model. Everything after step 8 is Weave's own record.

The two-phase agent pipeline

This is the core of the design, and the split is a security boundary before it is an engineering one. Isolation is not a prompt asking the model to stay in its lane — it is a separate session that has never been shown anyone else's data.



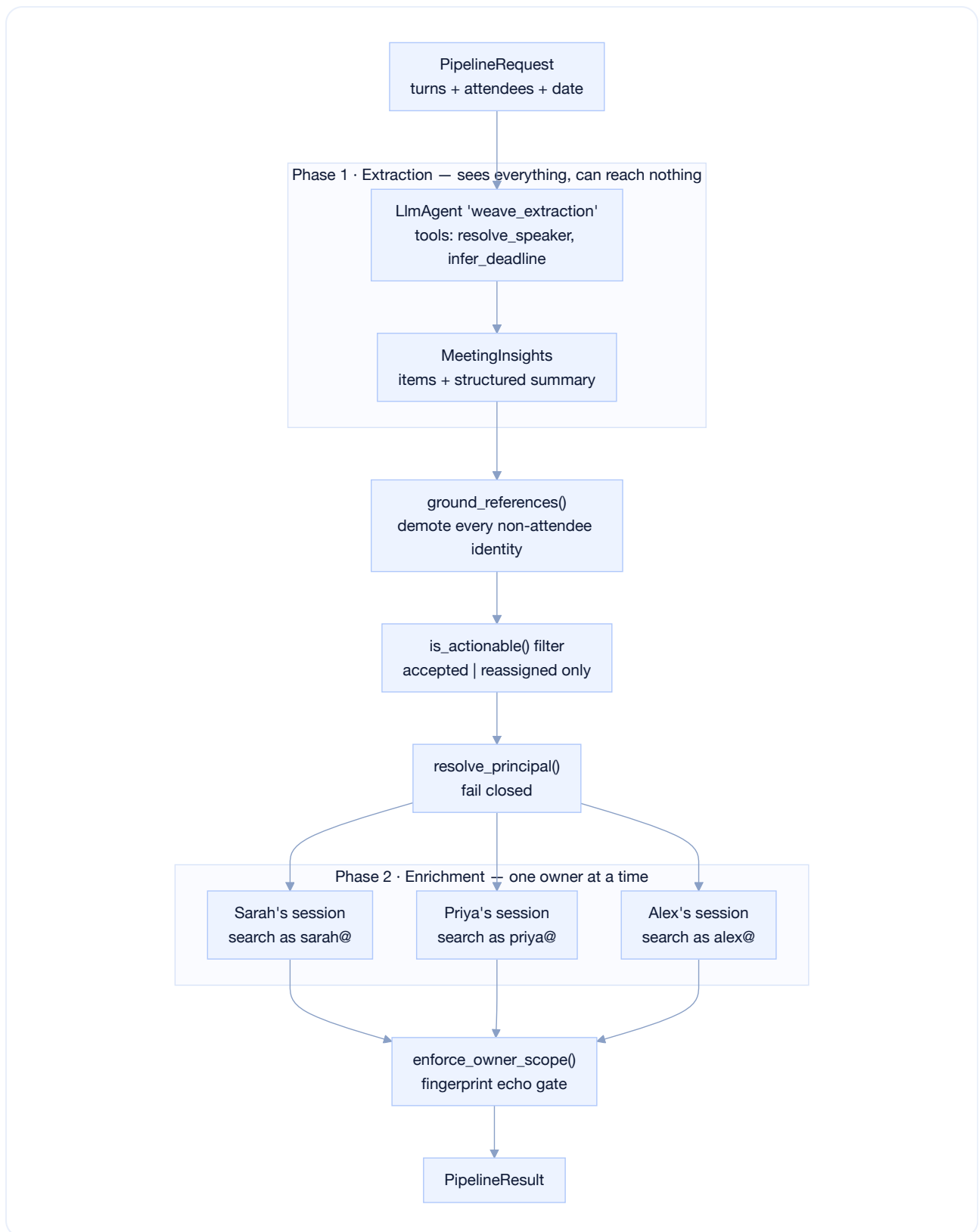
The capability boundary. Phase 1 sees every attendee's words and holds no retrieval capability whatsoever; phase 2 holds exactly one retrieval tool and sees one owner's items. No single reasoning step ever holds both, which is why a leak cannot be produced by the model misbehaving — `test_extraction_agent_has_no_context_tools` and `test_enrichment_session_state_contains_only_owner_items` pin both halves.

Phase 1 — extraction

One `LlmAgent` reads the entire transcript. Its tool list is the whole point:

TOOL	PURPOSE
<code>resolve_speaker</code>	Match a participant ID or spoken name against trusted Meet attendee state , returning an email and a confidence. Exact participant ID → 1.0; exact display name → 0.95; fuzzy first or full name → score × 0.9; ambiguous → 0.0 and no email.
<code>infer_deadline</code>	Turn a spoken phrase into a date relative to the meeting date, or nothing at all. It never guesses.

Extraction also produces the meeting's structured summary — overview, topics, decisions, implementation notes, reproduction steps — bounded by the schema and required to be transcript-grounded.



One call, two phases, three pure functions between them. Nothing owner-scoped begins until every identity has been re-grounded against the real attendee list, every non-actionable item has been dropped and counted, and a principal has been resolved — or refused.

Between the phases — the trust re-grounding

Three pure functions run before any owner-scoped work begins.

FUNCTION	WHAT IT DOES
<code>ground_references</code>	Every person-reference the model resolved is checked against the real Meet attendee list. Anything not present, or below the 0.85 confidence floor, is demoted to <code>status="unknown"</code> with its identity fields cleared. The spoken mention and turn index are kept, so provenance survives even when identity does not.
<code>is_actionable()</code>	Only <code>accepted</code> and <code>reassigned</code> items with a real owner continue. Everything else is counted into <code>dropped_item_count</code> — which is what distinguishes “found nothing” from “verified nothing”.
<code>resolve_principal</code>	Fails closed on three distinct grounds: no email, confidence below the floor, or an owner who was not an actual attendee. Each produces an unenriched bundle <i>carrying its reason</i> — never a search.

Phase 2 — enrichment

For each owner, a fresh `InMemoryRunner` session is created with `user_id` set to that owner's email, and session state containing *only* that owner's items. The agent has exactly one tool, `search_related_context`, which is read-only and constrained to the session's `search_principal`.

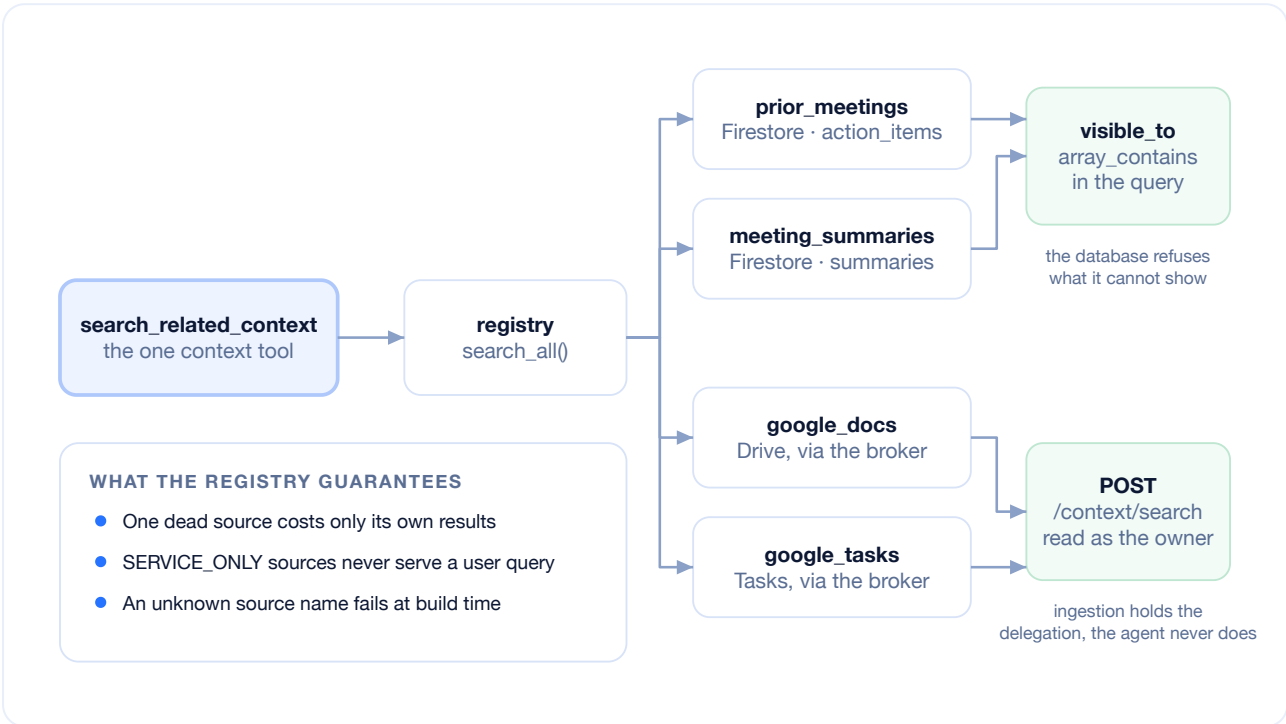
THE ECHO GATE

A model asked to return items back can quietly reword, duplicate, drop, or re-own them.

`enforce_owner_scope` refuses to trust the echo: every returned item is SHA-256 fingerprinted against the exact inputs, with multiplicity preserved. Only exact matches are accepted, and only the **original** item is kept — the model's copy is discarded, and only its additive fields (`title`, `details`, `matches`) survive. If every item fails the gate, the owner gets an unenriched bundle with `skip_reason="enrichment_echo_mismatch"` rather than a silently corrupted card.

The context framework

Enrichment's single tool fans out across a registry of owner-scoped sources. Adding a provider is one subclass and one config entry; the framework handles fan-out, failure containment, ranking and principal scoping.



Two enforcement shapes, one interface. Firestore-backed sources carry the ACL as a query predicate, so the database cannot return a document the principal may not see. Google-backed sources never call Google at all: they ask the ingestion broker, which holds the delegation and performs the read as the owner.

SOURCE	AUTH MODE	BACKEND	ACL ENFORCEMENT
prior_meetings	USER_CONTEXT	Firestore action_items	visible_to array_contains inside the query
meeting_summaries	USER_CONTEXT	Firestore meeting_summaries	Same, plus the current meeting excluded from its own context
google_docs	USER_CONTEXT	Drive, via the broker	Delegated read impersonating the owner
google_tasks	USER_CONTEXT	Tasks, via the broker	Delegated read impersonating the owner

The context broker

Agent Engine has **no domain-wide delegation, ever**. When enrichment needs a Drive file or a Google Task, it calls **POST /context/search** on ingestion with an OIDC token. Ingestion — the only identity holding broad delegation — performs the read *as the owner* and returns matches. The broker refuses a subject who is not onboarded, clamps the limit, truncates the query, rejects unknown sources, and **never logs the query text**: Google API errors can embed the request URL, so only the exception type is recorded.

Retrieval strategy

Both Firestore-backed sources use the same shape: **recency decides what enters the candidate window, relevance decides what leaves it**. A 40-document window is pulled under the ACL, ranked by vector similarity where embeddings exist, and falls back to IDF-weighted lexical cosine when the vector index or embedder is unavailable.

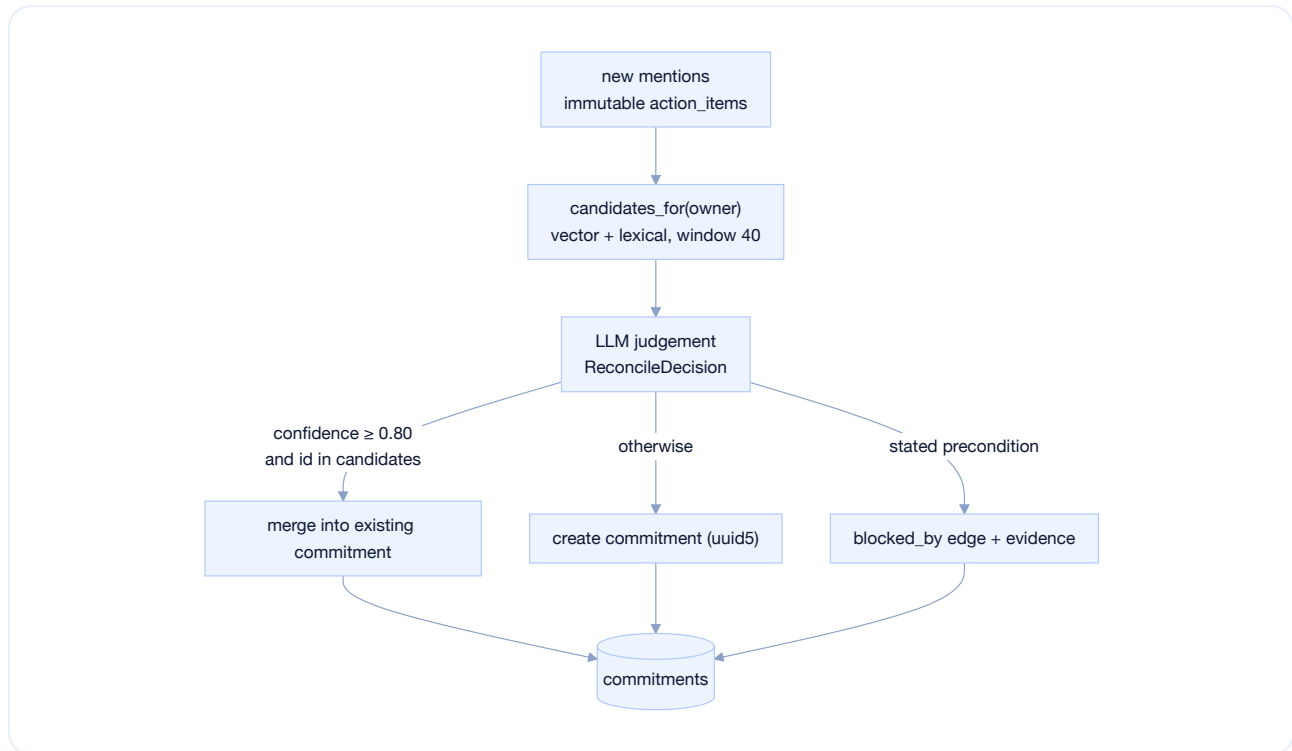
WHY THE BACKFILL IS NOT OPTIONAL CLEANUP

The lexical fallback keeps search *available*, which is exactly what makes a gap easy to miss — but a document with no `embedding` field is invisible to vector search. That is why backfilling embeddings is a required rollout step, and why the check is “no action-item document lacks the field”, not “search still works”.

IN ONE LINE One interface, four sources, two enforcement shapes — and no path by which a source can widen what its caller is allowed to see.

The commitment graph

Reconciliation runs in ingestion after persistence, converting immutable mentions into an owner-scoped graph. The same deliverable raised in three meetings becomes one commitment with a three-week history.



Every path that is not a confident, verifiable match creates a new node. A wrong merge hides a promise inside another commitment, where nothing will surface it again; a missed merge is at worst a duplicate the owner can see.

The judgement returns a structured `ReconcileDecision`: matched id, confidence, relationship (`original` · `restated` · `carried_over` · `progress_evidence` · `completion_evidence`), canonical title, inferred state, evidence, `waiting_on` and `blocking_hint`. Three guards make it safe:

- **Thresholds and membership are enforced in Python**, not trusted from the model. A match needs `confidence ≥ 0.80` and an id that was actually in the candidate list.
- **Edges require a stated precondition.** A hint naming a candidate outright is honoured directly; a hint naming something else is resolved by similarity at `≥ 0.80` or dropped. Shared topic, project or person creates nothing, and every edge stores the mention that stated it.
- **The judgement reads more than the description.** `judgement_text` assembles the tidied description plus the stated precondition, the spoken source text and the enrichment details — because a dependency usually survives in the spoken words and is paraphrased out of the description. Judging on the description alone is why the graph originally had no edges at all.
- **Inference stops at the lifecycle.** `likely_complete` may be inferred from completion evidence, with its confidence and evidence shown. `closed` never is — only a human closes a commitment.

The whole fold is one transaction over three documents — the commitment, the mention beneath it, and the action item's back-link — so a crash can never leave a commitment whose mention count disagrees with its mentions. Replaying a mention is a no-op.

Prioritisation

`weave_common/commitment_view.py` is presentation-neutral and shared by every surface, so a Chat card and a copilot sentence state the same facts for the same reasons. Ordering is arithmetic over stated facts, not a mood the model was in.



Urgency groups, in declaration order. A dated promise outranks an undated one, which is why both deadline checks come before blocking and waiting; `active` is the fallback, because no signal is not the same as nothing to do.

SIGNAL	CONTRIBUTION TO THE ATTENTION SCORE
Transitive open dependents	<code>× 200</code> each
Overdue	<code>+1000</code> plus days late, capped at 365
Waiting ≥ 7 days	<code>+500</code> plus days since the last mention
Open	<code>+10 ×</code> mention count, capped at 50 mentions
Likely complete	<code>-50</code>

The ratio is a product judgement stated once, in code: roughly five unblocked commitments equal one missed deadline. Change it there and every surface follows.

Each row is decorated with an `urgency_label`, a `reason` (why it surfaced), a `recommendation` (what to do — which differs, because “finish it” is useless advice for blocked work), `blocked_by_titles`, `open_dependents` and `carry_over`. The copilot is instructed to *report* these strings rather than compose its own rationale.

ABSENCE IS MODELLED, NOT RENDERED AROUND

Every optional field is `None` when the underlying fact is absent, deliberately: “show it only when relevant” is a property of the data, not a rule each renderer must remember. A row with no deadline can never be described as overdue.

IN ONE LINE Ordering is a pure function of stated facts, written once and read by every surface — so the answer is the same wherever you ask, and it can always say why.

One direct message carries the whole product

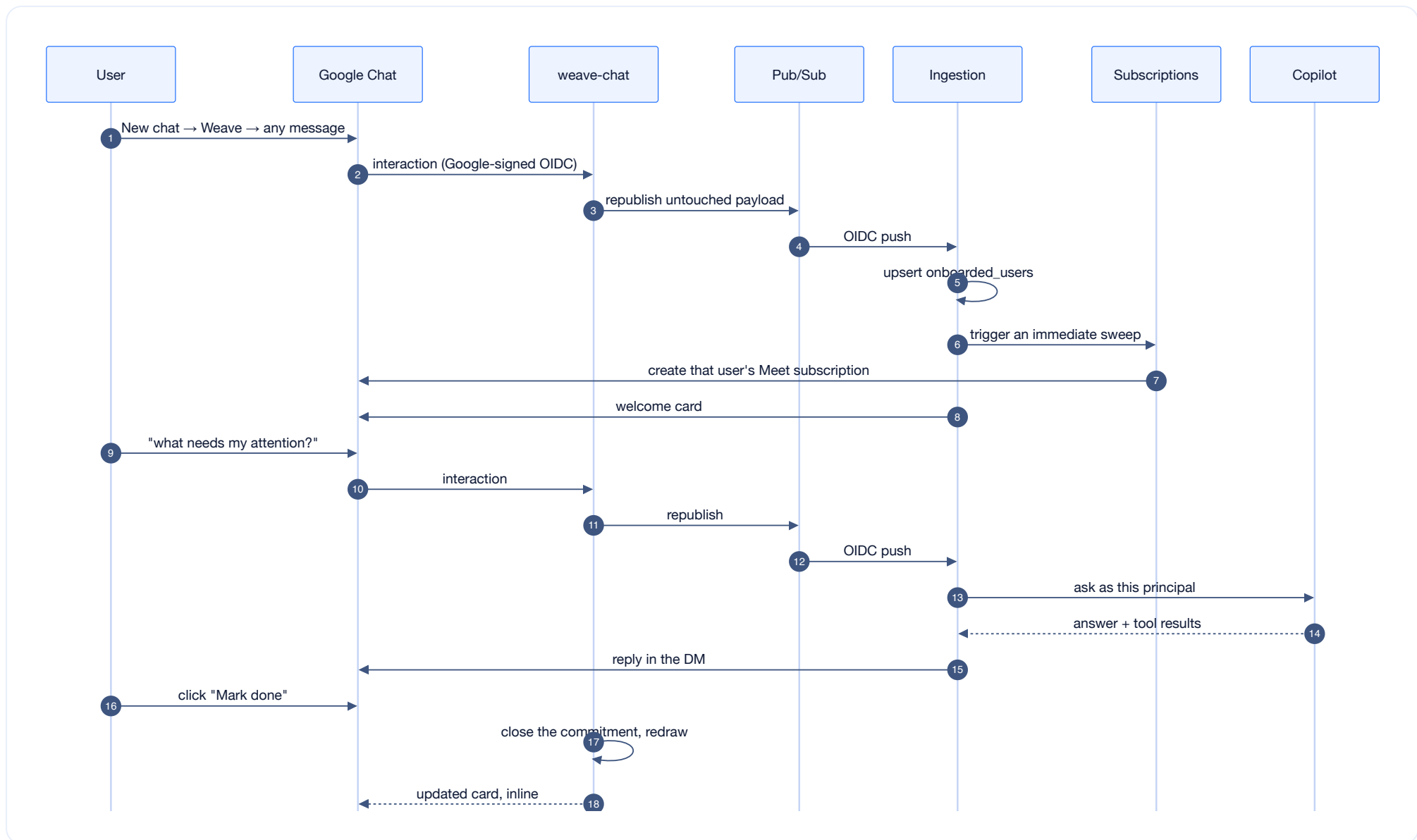
Enrolment, delivery and conversation happen in the same thread: a user adds Weave in Chat, their Meet subscription is provisioned for them, cards arrive there, and questions are answered there. The full interaction is overleaf.

Three details that carry the weight

- **Clicks are answered synchronously; everything else is republished.** Chat's interaction deadline is far shorter than a copilot turn, so `weave-chat` handles what it can inline and hands the untouched payload to ingestion for the rest. That also makes the surface reversible — pointing the Chat app's connection back at the Pub/Sub topic restores the previous behaviour with no redeploy.
- **The click payload carries a user id, never an address.** `weave-chat` looks the address up in `onboarded_users`, so a forged body cannot name somebody else's commitments — and every commitment read and lifecycle write is owner-guarded besides.
- **A card is redrawn from stored state.** After a close, the ids that were rendered are re-read under the owner guard and re-rendered, so the card can never assert something the record does not carry, and a row that is not the clicker's simply drops out.

DIRECT MESSAGES ONLY, BY CONSTRUCTION

“Join spaces and group conversations” stays off — which is also why a direct-message install arrives as a `MESSAGE` event, the onboarding signal the handler acts on. Nothing Weave produces is ever posted where somebody else can read it.



Install, delivery and conversation, in one thread. Steps 1–8 are onboarding and run once; steps 9–15 are an ordinary question; steps 16–19 are a button press, answered inline in the two seconds Chat allows.

Interfaces, topology and packages

Ingestion service surface

ENDPOINT	CALLER	AUTH
POST /pubsub-push	Pub/Sub (meet-artifacts)	OIDC, aud = service URL, verified SA
POST /context/search	Agent Engine	OIDC from the agent service account
POST /chat-events	Chat events topic	OIDC from the push service account
GET /healthz	anyone	none

Cloud Run only accepts an OIDC `aud` equal to the service URL, which is why `custom_audiences` is declared on the service — without it, push 401s at the platform layer and never reaches the container.

Transcript ingestion routes

MODE	SOURCE	REQUIREMENTS
live	Meet REST API v2, read as the subscribing user	Domain-wide delegation and the org-policy exception
fixture	Bundled JSON under <code>services/ingestion/fixtures/</code>	None beyond the project

The fixture route exists so the deployed pipeline can be exercised end to end — screening, extraction, ACLs, reconciliation, delivery — before any Workspace wiring exists, and so a smoke test after every rollout costs one `gcloud pubsub topics publish`. In `live` mode the meeting's agenda-like title comes from the transcript document's Drive filename; that read is best-effort, so a card keeps the meeting time and omits the title rather than failing.

Deployment topology

COMPONENT	RUNTIME	IDENTITY	NOTABLE
<code>weave-ingestion</code>	Cloud Run	<code>weave-ingestion-sa</code>	The only identity with broad domain-wide delegation
<code>weave-chat</code>	Cloud Run (public)	<code>weave-chat-sa</code>	Answers card clicks inline; no delegation
<code>weave-subscription-manager</code>	Cloud Run job + Scheduler	<code>weave-subscriptions-sa</code>	Delegation for the Meet scope only
Pipeline engine	Agent Engine	<code>weave-agent-sa</code>	No delegation, ever
Copilot engine	Agent Engine	<code>weave-agent-sa</code>	Same runtime identity, separate deployment

Delegation is keyless throughout: service accounts sign their own assertions through the IAM Credentials API (`roles/iam.serviceAccountTokenCreator` on themselves) rather than carrying exported private keys. Both engines are deployed by `make deploy-agent` and `make deploy-copilot`, which update the existing deployment so the resource name stays stable for everything that references it.

Package layout

PACKAGE	DEPENDS ON	ROLE
<code>weave_common</code>	pydantic only	The contract. Importable everywhere, including inside Agent Engine. Nothing heavy, ever.
<code>agent</code>	<code>weave_common</code> , <code>google-adk</code>	Agents, prompts, tools, context sources, identity, deployment
<code>weave_ingestion</code>	<code>weave_common</code> , Google APIs	Pipeline handler, broker, persistence, reconciliation, delivery, Chat events
<code>weave_chat</code>	<code>weave_common</code> , <code>weave_ingestion</code>	Chat interaction endpoint and inline card actions
<code>weave_subscriptions</code>	<code>weave_ingestion</code>	Subscription lifecycle job

Cross-package data always crosses as a `weave_common` pydantic model — never an ad-hoc dict. Most models are `frozen=True`, `extra="forbid"`, so an unexpected field is an error at the boundary rather than a surprise three layers in.

Weave · Architecture. Generated from `docs/ARCHITECTURE.md` and the diagram sources in `docs/architecture-pdf/`.

Companion documents: the security model (invariants, identity, ACLs, threat notes), the data model (collections, indexes, identifiers) and the deployment guide. Rebuild with `make -C docs/architecture-pdf`.